

Group Members and Names

Class: COMP 479; Dr. Manar Mohaisen Fall 2025

Final Project ADR

Github Repo: <https://github.com/cacayan2/classifier-DoS-project>

- **Emil Cacacayan**
 - LUC UID: ecacayan
 - LUC Student ID: 00001521140
- **Umar Siddiqui**
 - LUC UID: usiddiqui1
 - LUC Student ID: 00001519781
- **Nafisa Sabir**
 - LUC UID: nsabir
 - LUC Student ID: 00001729292
- **Fnu Syed Moosa Aleem “Moosa”**
 - LUC UID: fsyedmoosaaleem
 - LUC Student ID: 00001704732

1. Executive Summary and Project Goals

The goal of this project is to design, train, and evaluate machine learning classifiers that distinguish DoS (denial of service) attacks from benign traffic in the CIC-IDS2017 dataset using Wednesday traffic. DoS attacks are a robustly documented threat in cybersecurity, and there are many tools deployed on the market designed to detect and intervene such attacks. DoS attacks aim to overwhelm some network or service by sending unusually high volumes of traffic or exploiting weaknesses in the protocol, preventing users from accessing the system, hence denial of service (Kumar, Singh, & Sharma, 2020). The motivation for detecting such attacks is primarily financially and security driven: attacks can disrupt business operations and issues with client privacy, and detecting them promptly is a problem with practical and logistical implications.

This project aims to emulate as closely as possible the machine learning project paradigm introduced in COMP 479, including ETL, exploratory data analysis, preprocessing, model training, evaluation, and deployment of the best performing model as an API service. Although we may dwell on the semantics regarding the technical components, at a very high level the overall purpose is to demonstrate a test-driven, well-documented, and collaborative approach for first identifying harmful traffic problems and at large implementing, testing, and deploying machine learning models. This design is tailored (both by design by our instructor and within the group) to represent how ML models may be developed and deployed in an enterprise setting. The documentation is created in such a way that stakeholders, data engineers, and machine learning developers understand motivation, workflow, and results.

This dataset we are utilizing to both train and test the models were obtained from a well-known intrusion detection benchmark which was ultimately designed to emulate real network behavior. It contains labeled activity (benign and categories of DoS attacks) (Sharafaldin, Lashkari, & Ghorbani, 2018). Due to the rapidly evolving nature of cyberattacks, these benchmarks are mostly useful for the sake of demonstration in academic and instructional environments where labeled data is needed to experiment and develop models.

Technically speaking, the goal of this project is to build a classification system that can distinguish between normal network traffic and different types of DoS attacks. To this end, we follow these steps: - Loading

and validating raw data - Standardizing column names and formats - Cleaning and transforming data into a consistent format - Exploring relationships and distributions of features - Training a several classification models - Evaluating model performance, and lastly - Preparing the best model for deployment

At each step, we document decision-making and justifications and emphasize the latter. This ADR (Architectural Decision Record) structure ensures that every design choice is explained in appropriate context and justified accordingly. This makes the project transparent and increases maintainability and portability of any code/choices we make, while allowing others to follow reasoning behind each step closely.

From an economic perspective, detecting these attacks ultimately support business goals. Especially for businesses that offer online services, being unable to provide these services can cause major losses in revenue and customer trust. In practice, abstraction of many of the ideas discussed in machine learning development (and vice-versa to machine learning engineers) are not required to be understood by business owners and other clients, but as stated earlier the interest comes from the more abstract outcomes - systems that are able to reliably flag traffic as suspicious. Such a tool can help guide manual investigations - something made more robust with structured analysis and careful handling of datasets. This reinforces the need for thoughtful data engineering choices and practices in such applications.

From a cybersecurity perspective, detecting DoS attacks offers quite a few benefits. The ability to identify such attacks early can help limit service interruptions and reduce impact of an attack. Secondly, recognizing different types of activity allows security teams to design interventions more specifically (Rauf et al., 2021). Thirdly, collecting and analyzing traffic data helps networks establish a baseline and compare with future data to determine if a traffic flow contains the fingerprint of malicious transfers.

Another important project goal is clear communication and collaboration. This is a group project, and so early on in the project design, expertise and interest were expressed by the members of the group. This repository’s structure, project board, and communication expectations are designed to help the team maintain organization while ensuring that deliverables are submitted in a timely manner. Each notebook or script serves a purpose, and every team member is expected to record any findings, developments, or decisions in a shared and visible format. This reinforces the way collaboration is performed in the real world where teams coordinate across parts of an even larger workflow.

Finally, we want to develop tools and deployment in a way that fosters reproducibility, maintainability, and code portability. In this project, we also outline a schema file describing the final structure of the cleaned dataset so that both end-users and developers forking off the project can apply the same manipulations to their own datasets consistently. This is essential for maintaining reliable data pipelines in production-level environments (Breck et al., 2019). Even though project does not involve full production deployment, preparing an API skeleton and defining data requirements is a fundamental part of this development process and helps reinforce all previously mentioned points.

The project goals can be divided into 5 parts: 1. Build an end-to-end and well-documented pipeline for detecting adverse network traffic. 2. Demonstrate collaborative best practices and defining clear roles, structured documentation, and clear lines of communication. 3. Produce interpretable results that help understand the data/ML engineering relevance and the data-driven patterns underlying denial of service attacks. 4. Prepare a foundation for a deployable system. 5. Prioritize clarity over speed and complexity so that the final product is easily evaluated by both the team and outside audience.

2. Data Sources

2.1 Context and Technology Choice

The primary data source for this project is a publicly available intrusion detection dataset developed by the Canadian Institute for Cybersecurity (CIC). The dataset is available on the following website: <https://www.unb.ca/cic/datasets/ddos-2019.html> and contains labeled network traffic representing normal activity as well as several different types of cyberattacks (the main focus being DoS attacks). This dataset was

generated using realistic traffic patterns and controlled attack scenarios in order to capture the behavior of true network systems (Sharafaldin et al., 2018).

The reason for selecting this dataset is twofold - the first is that it is widely recognized within the realms of cybersecurity and intrusion detection as a high-quality benchmark (Ring et al., 2019). In addition, attack classes outlined by the assignment include GoldenEye, Hulk, slowhttptest, slowloris, and Heartbleed allow for a more heterogeneous exploration of binary classification. For this assignment, the data was provided in a few different formats (.csv, .json, .parquet) so that we had the opportunity to demonstrate different loading techniques and practice with ETL while utilizing different file types.

2.2 Justifications

This dataset is appropriate for the goals of the project for a few reasons:

1. The dataset is highly relevant for modern DoS behavior

This dataset includes many different types of DoS attacks along with different strategies, which actually line up pretty well with the descriptions about how DoS attacks have evolved over time (Kumar et al., 2020).

2. Acceptance of the dataset as a benchmark within cybersecurity and machine learning research

A few surveys identify this dataset and the rest of its family (CIC-IDS) as one of the more commonly used and meticulously designed options for experimentation (Ring et al., 2019). This allows for work to be easily comparable with other similar research projects.

3. Availability of labels

This dataset includes well-defined labels for every type of traffic - this lends itself to supervised learning and allows for easier evaluation of models.

4. Practical for ETL

Because the dataset is fairly large and the files provided to us are quite diverse, it provides an environment for demonstrating real data engineering tasks.

5. Alignment with assignment requirements

Although this is self-explanatory, it's important to note - this is the dataset outlined specifically by the project requirements.

2.3 Status

Status: Accepted

Date: November 26, 2025

Team Members: Emil Cacayan, Umar Siddiqui, Nafisa Sabir, Fnu Syed Moosa Aleem “Moosa”

3. Data Integration (Extract)

3.1 Context and Technology Choice

The goal of the integration/extract phase of this project is to bring together the raw files provided (a collection of `.csv`, `.parquet`, and `.json` files) from the aforementioned CICIDS dataset and load the files into memory so they can be further digested by univariate and multivariate exploratory data analysis and subsequent model training and testing. To this end we utilized Python libraries with data loading tools (mainly `pandas.read_csv()`, `json.load()`, and `pyarrow.parquet.read_table()`).

This extraction was coded into the script `etl/load_files.py` and documented in `notebooks/00_etl_exploration.ipynb`. The preprocessed data was saved into `data/interm/combined_raw.csv`. This represents the data that will be used for remaining downstream analyses/applications.

This process follows ETL processes generally use for most ML applications and the model introduced in class involving the processing of raw data from multiple sources and unifying into a single dataframe or data representation that is preserved prior to subsequent cleaning or data transformation.

3.2 Justifications

Python and its libraries allow for a paradigm that lends itself to big data processing and machine learning applications. The tools outlined in the context and technology section allowed us to stitch together different data sources into a single dataset that matches the intended structure of the dataset. In particular, the `pandas` module contains different modules that have been well-documented and tested for its ability to handle, merge, and concatenate large structured datasets. Integrating the data into a single file allows for consistent data representation in downstream applications and serves as a checkpoint so that any transformations of the dataset can be traced/rolled back to this intermediate state.

3.3 Status

Status: Accepted

Date: November 29, 2025

Team Members: Nafisa Sabir (designated ETL lead), agreed upon with Emil Cacayan, Umar Siddiqui, and Fnu Syed Moosa Aleem “Moosa”

4. Data Transformation (Transform)

4.1 Context and Technology Choice

In the transformation step, the data was cleaned and standardized to increase fidelity and reproducibility of downstream transformations and training. This process includes dropping unnecessary (such as index) columns or duplicates, implementing consistent column naming patterns, and filtering out particular labels (in this case, `Heartbleed` was filtered out due to not being a DoS attack, which models downstream will be trained to discriminate).

All of these operations are performed in the script `etl/clean_dataset.py` and documented in `notebooks/00_etl_exploration.ipynb`. This cleaned dataset is saved in `data/cleaned/wednesday_clean.csv`, the naming is due to the fact that this dataset (theoretically) contains label from Wednesday traffic, since the Wednesday traffic flows contain the labels required for training models. Lastly, to ensure this classification task remains binary and not categorical, the different DoS attacks were encoded to binary 0/1 (representing benign or attack respectively).

In other words,

$\text{Label}(\text{Wednesday}) \in \{\text{DoS GoldenEye}, \text{DoS Hulk}, \text{DoS slowloris}, \text{DoS Heartbleed}, \text{Benign}\}$

4.2 Justifications

This step is necessary because the raw dataset contains inconsistent column naming conventions across the different file types - testing of this dataset resulted in the failure of downstream processes (creation of duplicate columns, etc.). This could also result in ambiguous columns/features. In addition, numeric fields load as objects/strings - to ensure that transformations can be applied to these numeric features, some must be directly typecasted to float/integer data types. In addition, this assignment is a binary classification task for detecting denial of service attacks - because **Heartbleed** is not a DoS attack, it must be removed, and the remaining features must be encoded to fit the binary classification task. Abstracting this task in code is required for reproducibility internally within the team and with end clients, users, or developers.

4.3 Status

Status: Accepted

Date: November 29, 2025

Team Members: Nafisa Sabir (designated ETL lead), agreed upon with Emil Cacayan, Umar Siddiqui, and Fnu Syed Moosa Aleem “Moosa”

5. Data Storage (Load)

5.1 Context and Technology Choice

The processed dataset is stored in a csv file in `data/final/wednesday_clean.csv` and is loaded in the script `etl/load_files.py` and documented in `notebooks/00_etl_exploration.ipynb`. This file represents the cleaned, de-duplicated, and type-corrected dataset. The CSV format was selected because it is portable, human readable, and easily ingestible in other Python workflows (such as `pandas`). We use the library’s `read_csv` method to load the data into a `DataFrame`. This operation loads the full dataset into memory and preserves all numeric and categorical features required for schema generation, preprocessing, feature engineering, exploratory analysis, and model training. Because the dataset is not too large, no complex storage management systems were required.

5.2 Justifications

1. **CSV ensures transparency and easy debugging.**
2. ****File size is manageable allowing for faster read times.**
3. **No nested or multi-table structure exist - data is stored flat.**
4. ****Dataset is reference consistently across notebooks and scripts, ensuring reproducibility. This dataset serves as canonical truth for other downstream components.**

5.3 Status

Status: Accepted **Date:** November 29, 2025 **Team Members:** Nafisa Sabir (designated ETL lead), agreed upon with Emil Cacayan, Umar Siddiqui, and Fnu Syed Moosa Aleem “Moosa”

6. Reading Data

6.1 Context and Technology Choice

Part of the pipeline requires a reliable and replicable workflow for reading data for our classifier model to utilize. After ETL is completed and a cleaned dataset is produced, the next challenge to overcome is ensuring both training and deployment environments are configured to read data in the same way. Small inconsistencies (mismatched column order, unexpected data types, or renamed features) can cause significant model failures during inference. To prevent this, a schema file was defined and implemented and stored as `data/final_features.json`. This schema file defines the following: - Expected feature names - The data types of each feature - Input validation rules - Order in which the features must appear, and - Any preprocessing statistics needed (especially those required for scaling such as means or standard deviation).

`.json` was selected for the schema format - it is lightweight, widely supported across different languages, and is easily readable within Python. This schema ensures compatibility between our training and deployment scripts, and we will be utilizing the FastAPI service that would load the model and user-submitted feature vectors.

In terms of implementation, we utilize Python's built-in `json` module for reading and parsing the schema during preprocessing. Within the deployment pipeline (`deployment/schema_loader.py`), the `.json` schema is loaded and used to verify incoming API requests before passing them into the trained classifier.

Adopting a formal schema is generally considered best practices for machine learning development and requires explicit and strict data contracts to prevent failures and components receive consistent and well-defined inputs (van der Aalst, 2022).

6.2 Justifications

There are quite a few reasons why the use of a schema is not only recommended but essential for the success of this project:

1. Consistency Between Training and Deployment

ML models, especially in this application for structured network traffic data, generally require feature alignment. Order and types of the features must be consistent between training or inference or else the model will not behave predictably and may produce incorrect predictions. Our schema guarantees that the model receives data identical in structure to the data it was trained on (Khaitan & McCalley, 2015).

2. Preventing Data Drift and Input Errors

Network traffic datasets are naturally vulnerable to irregularity - many datasets or vectors that may pass through a deployed model may have missing values, unexpected labels, or inconsistent units especially when pooling from multiple different data sources (which is something which was encountered with the dataset used in this project). Using the schema to validate data passed to the model serves as protection and ensures that any and all inputs conform to expected format before inference. This in turn reduces risk of runtime errors and makes the final pipeline more robust (Breck et al., 2017).

3. Maintainability and Internal Collaboration

Multiple team members are involved in handling the ETL, modeling, and deployment within this project. The use of a schema explicitly defines expectations and reduces confusion between the format of dataset used for training and inference. This is especially important in this case where there are multiple notebooks and scripts to manage - this helps reduce the logistical overhead involved with manual coordination (which also lends itself to being error prone).

4. Compatibility with FastAPI

FastAPI supports validation frameworks (such as Pydantic among others) that are able to utilize schema definitions to explicitly enforce the correctness of passed data. Storing the schema in `.json` format allows for seamless integration with these validation tools. This improves the reliability of our API and reduces the chances of incorrect or malformed requests causing runtime error or logical failures (Tiangolo, 2018).

5. Reproducibility and Transparency

Across scientific, machine learning, and intrusion detection research, reproducibility is a requirement. A schema allows future users - groups, instructors, testing, or developers - to have a consistent and definitive reference describing the correct inputs expected by the classifier. Should new data be available for this classifier to be read, retraining and evaluation is relatively trivial compared to having to reverse engineer or carefully document (and follow said documentation) the expected data for the model (i.e., passing incorrectly formatted data to the model *could* work, but that does not mean that it is correct - a logical error) (Ring et al., 2019).

6.3 Status

Status: Accepted

Date: December 1, 2025

Team Members: Nafisa Sabir, Emil Cacayan, Umar Siddiqui, and Fnu Syed Moosa Aleem “Moosa”

7. Exploratory Data Analysis

7.1 Context and Technology Choice

The purpose of univariate exploratory data analysis (EDA) is to understand distributional properties of features in a dataset prior to exploring relationships between two variables (bivariate analysis) or across multiple features (multivariate analysis). This step identifies inconsistencies, anomalies, outliers, and sources of bias - downstream transformations and training must rely on robust, meaningful, and interpretable features. Network intrusion often exhibit skewed numerical distributions, heavy-tailed traffic features, or artifact that is inherently present from the somewhat inconsistent sampling of traffic data (i.e. we must address missingness), it is essential that features be examined early and independently prior to building classification models (Ring et al., 2019).

Our team utilized Python, Jupyter notebooks, the Python module `pandas`, `numpy`, and `matplotlib/seaborn` for variety of univariate EDA related tasks. These tools were chosen because of their efficiency in processing big data, intuitive visualization capability, and historically being well-documented and tested. All analysis for this stage is stored in the notebook `notebooks/01_univariate_eda.ipynb` and the cleaned dataset used in this analysis can be found in `data/cleaned/wednesday_clean.csv`.

Bivariate analysis evaluates how different pairs of features relate to one another focusing on correlation patterns and identifying/addressing multicollinearity concerns within the cleaned Wednesday dataset. Understanding these relationships is essential for intrusion detection because many network flow features tend to co-vary in predictable ways during benign or attack service flows (for instance, packet rate often increases/decreases simultaneously with flow byte count or header size) (Kumar et al., 2020).

Similar packages were utilized in the bivariate analysis as the univariate analysis: - `pandas` was selected to calculate paired statistics and correlation coefficients - `numpy` was used for matrix operations and manipulations - `seaborn` and `matplotlib` were used for visualizations (more specifically to generate correlation heatmaps and selected pairplots)

Pearson’s correlation coefficient was used to measure linear relationships between the numerical variables. This is common practice in network data analysis (Ring et al., 2019). The heatmap provides a macroscopic view of high-correlation features, while the pairplots allow closer inspection of pairs with suspected collinearity or features that possess higher class discriminatory power.

Because our dataset consists primarily of numeric network flow features, these visualization techniques are appropriate and computationally efficient. They also aid in interpretability for downstream decision-making in an otherwise very skewed and highly dimensional dataset.

The bivariate EDA was completed in `notebooks/02_bivariate_eda.ipynb` by Umar and Moosa. Initial correlation heatmaps and pairplots have been generated, with full results to be finalized as the team completes assigned tasks. Identified collinear features and observed class-separation patterns will help guide preprocessing decisions, model selection, and feature handling in subsequent stages of the pipeline.

The multivariate analysis expands upon the earlier work by examining structure of the dataset across multiple features simultaneously. This assists in the identification of high-dimensional patterns not visible in univariate or bivariate analysis. Two primary techniques were selected:

1. Principal Component Analysis (PCA)

This was used to project the 78-feature dataset into a lower-dimensional space across axes capturing the highest variation to identify features that dominate variation and explore whether benign and DoS traffic exhibit separable structure in aggregated feature combinations. This is widely used in intrusion-detection research and machine learning at large to understand structure in highly dimensional datasets (Ring et al., 2019; Sharafaldin et al., 2018).

2. K-Means Clustering

This method was utilized not for its power as a classifier (since it assumes numbers of clusters as a hyperparameter and highly spherical data, which this dataset likely violates), but is used as a preliminary assessment for determining if the data naturally forms groupings that resemble the attack and benign labelling. Clustering provides insight into whether the dataset contains meaningful high-dimensional separation and can reveal mislabeled, ambiguous, or outlier samples (Kumar et al., 2020).

These analyses were performed in Python using `scikit-learn`, which is considered part of a standard workflow in modern intrusion detection and other machine learning pipelines. PCA aids visualization and informs potential dimensionality-reduction decisions, while clustering helps evaluate separability and structural consistency before model training.

Multivariate analysis was completed in `notebooks/03_multivariate_eda.ipynb`, with Nafisa leading PCA and K-means implementation and Moosa producing corresponding visualizations. Preliminary PCA results show identifiable variance structure, and clustering procedures will be further evaluated for alignment with true attack labels. Full results will be finalized and incorporated into the modeling decisions in subsequent steps of this machine learning development paradigm.

7.2 Justifications

As mentioned previously, univariate EDA is essential for later preprocessing and modeling decisions for the following reasons:

1. Identification of Outliers

Many network traffic features demonstrated having very long-tailed distributions - particularly the features having to do with flow duration, packet counts, and byte rates. Univariate EDA assists in identifying outliers

that could distort scaling, central tendency statistics, or cause unstable model behavior. Our professor recommended use of the interquartile range method (IQR) for outlier detection and filtering, which we utilized.

2. Detection of Low-Variance or Zero-Variance Features

As many academics in statistics courses and papers have stated in the past, variance is information. Features with minimal variance provide little information of utility to a classifier and should be removed or at the very least deprioritized for modeling. Methods such as k-nearest neighbors or SVM's are not robust to features with low or zero-variance particularly when scaling is applied.

3. Verification of Data Types and Statistical Properties

Summary statistics (central tendency such as mean and median and dispersion such as standard deviation and range) were computed to ensure fidelity was not lost for features originally in object type following conversion to numeric. This step is important as occasionally datasets network contain malformed numerical fields with artifact stemming from improper structuring, formatting, or invalid characters (Sharafaldin et al., 2018).

4. Detection of Skewed Distributions

Highly imbalanced features should be scaled or transformed and may require adjustments in evaluation approaches. Skew is common in these types of datasets because benign behavior dominates most distributions (Kumar et al., 2020).

5. Preparation for Downstream EDA and Modeling

These preliminary evaluations help inform which features may later require special handling during normalization, standardization, transformation, feature selection, or modeling. This is generally accepted to be machine learning best practice, which emphasize detection of anomalies and artifact to reduce error propagation (Huyen, 2022).

Several motivations support inclusion of bivariate analysis for this project:

1. Identification of Multicollinear Features

Intrusion detection datasets often contain engineered or derivative features (i.e. they are some combination of other features) and encode similar phenomena (Sharafaldin et al., 2018). These highly correlated pairs may introduce redundancy into models (e.g. logistic regression, SVM) which causes unnecessary increased computational overhead, instability, or biased (unnecessarily large) coefficients. This analysis helps detect multicollinearity risks, guide decisions on whether dimensionality reduction is appropriate, and highlight which groups of features carry overlapping information.

2. Understanding Class Separation Structure

This whole project assumes there is some underlying structure within the 78 features that allows for discrimination between two class labels - the goal of this project is to capture said structure within a model and use it to classify new data (benign vs. DoS attack). In this context, this is important because attack flow characteristically display abnormal packet timings, payload sizes, and header behaviors, usually presenting as some overexpression of these features (Kumar et al., 2020). If certain pairs show clear separation, these relationships can inform model selection and hyperparameter choices.

3. Improving Interpretability and Model Robustness

Examining relationships between features early helps avoid downstream issues such as models overfitting on tightly correlated features, instability in coefficient-based classifiers (coefficients becoming inflated), or skewed decision boundaries when multiple features capture the same underlying signal. As previous work states and demonstrates, in intrusion detection datasets early EDA improves transparency and reduces errors in downstream modeling decisions (Ring et al., 2019).

4. Preparation for Decisions on Scaling and Preprocessing

Correlation analysis provides evidence supporting or refuting normalization or standardization methods during modeling. A dataset that possesses features that vary widely in scale or demonstrate strong covariation ultimately justifies the use of standardization and/or normalization during preprocessing to ensure that the model is trained on a balanced dataset.

The multivariate paradigm that we implemented was chosen for the following reasons:

1. Understanding High Dimensional Patterns

Network intrusion data typically contains dozens of engineered numerical features that capture variations in packet size, timing, flow duration, and header characteristics. These variables often interact in complex ways during attacks (Kumar et al., 2020). This analysis reveals whether attack behavior differs in aggregate structure compared to benign traffic, whether groups of features jointly contribute to identifiable variations in the dataset, and whether the dataset exhibits redundancy consistent with strong feature correlations detected earlier. This is important because redundancy in highly dimensional datasets will affect downstream model stability and will provide justification for dimensionality reduction.

2. Informing Model Decisions

PCA visualization indicates whether the dataset contains clear separation between classes in the axes of greatest variation, overlapping clusters that may demonstrate non-linear separability, and axes of variance dominated by noise or redundancy. This information is relevant to model selection and hyperparameter in downstream steps of the pipeline.

3. Evaluating Natural Grouping Behavior

Perhaps the data naturally groups into spherical clusters for benign vs. attack labels. K-Means clustering as an unsupervised learning method helps evaluate whether benign and attack traffic naturally separate in the absence of labels. Prior work demonstrated that DoS traffic often forms dense, distinct clusters due to the extreme, sustained packet characteristics associated with attacks (Sharafaldin et al., 2018). If clustering aligns with these labels, this provides evidence for dataset consistency and separability.

4. Supporting Downstream Preprocessing

Findings from PCA and clustering guide later stages with information on whether scaling is appropriate and which features to scale, whether dimensionality reduction is beneficial, whether certain groups of features contribute disproportionately to the variance of the dataset, and information on how to interpret model-training and inference results based on structural characteristics. These decisions improve both interpretability and performance of the final models.

7.3 Status

Status: Accepted (Univariate Analysis)

Date: December 1, 2025

Team Members: Fnu Syed Moosa Aleem “Moosa” (histogram and boxplot generation, outlier detection), Umar Siddiqui (summary statistics, low-variance feature identification, skew investigation), Nafisa Sabir (assisted Moosa with figure saving and summarizing outliers), Emil Cacayan

Status: Accepted (Bivariate Analysis)

Date: December 2, 2025

Team Members: Fnu Syed Moosa Aleem “Moosa”, Umar Siddiqui, Nafisa Sabir, Emil Cacayan

Status: Accepted (Multivariate Analysis)

Date: December 3, 2025

Team Members: Fnu Syed Moosa Aleem “Moosa”, Umar Siddiqui, Nafisa Sabir, Emil Cacayan

8. Data Preprocessing

8.1 Context and Technology Choice

The goal of this preprocessing stage is to make sure that all features are clean, numeric, consistent, and standardized before training. The original dataset contains mixed formats, outliers, and issues with scale - common in this type of network traffic dataset. A structured preprocessing pipeline was therefore warranted.

The preprocessing pipeline includes two stages - one performed prior to training, the other performed prior to deployment. In the training preprocessing implemented in the ETL and EDA notebooks, cleaning steps included standardizing feature names, removing non-numeric artifacts, verifying correct number of features, validating label categories, and producing a binary label. Final feature scaling statistics were computed from this trained cleaned dataset. In terms of deployment, this was implemented in the script `deployment/preprocessing.py` using the same steps. The main difference is that this process is largely codified in the `/data/final_features.json` file. At inference time, the system validates incoming samples, ensures they contain all required features, enforces numeric types, applies transformations, and arranges values in the correct order expected by the model.

This approach follows the principle that preprocessing pipelines should be consistent between training and production environments to prevent “training-serving skew” (Zaharia et al., 2018). All transformations under this paradigm are deterministic and reproducible.

All preprocessing logic required has been documented in the ETL and EDA notebooks. The deployment-time preprocessing pipeline has been fully implemented in `deployment/preprocessing.py` and validated using edge-cases in `notebooks/06_deployment_tests.ipynb`.

8.2 Justifications

A few motivations guided the design of this preprocessing pipeline:

1. **Consistency and Reproducibility** One of the big caveats in network intrusion datasets is reproducibility across different stages (Sharafaldin et al., 2018). By centralizing feature metadata and processing logic in a schema file instead of scattering the process across scripts and notebooks (particularly in a large project like this with multiple collaborators), we can prevent issues with feature order, scaling parameters, and input validation.
2. **Standardization** The team decided upon the design of the preprocessing pipeline prior to model evaluation, and it was understood that some of the algorithms tested in this project were not robust

against highly variable feature scale. Standardizing features improves stability and produces better convergence behavior (Pedregosa et al., 2011). Although tree-based models are scale-invariant, adding a scaling step to the preprocessing pipeline keeps the models more interpretable by ensuring consistency without affecting their performance.

3. **Robust Error Handling** Because this project involves cybersecurity, there is a risk of malformed input data. This pipeline includes explicit missing feature checks, extra feature checks, data type enforcement, and null type rejection. Prior literature emphasizes the use of strict input validation to prevent model misuse (Sommer & Paxson, 2010).
4. **Separation of Tasks** Separating preprocessing from model training and inference allows for a more efficient workflow. The team was more or less able to work independently on different parts of the project, assisting in streamlining the development of the project timeline.

8.3 Status

Status: Accepted

Team Members: Fnu Syed Moosa Aleem “Moosa”, Umar Siddiqui, Nafisa Sabir, Emil Cacayan

Date: December 4, 2025

9. Feature Engineering

9.1 Context and Technology Choice

In this project, feature engineering refers to the sum set of operations used to prepare raw data (network traffic) into meaningful features that can be used to train machine learning models. The CIC-IDS2017 dataset provides a comprehensive set of flow-based features (Sharafaldin et al., 2018), so the team decided not to create new transformations. Instead, core feature selection comes from the work done in EDA. Highly collinear features were removed since they do not provide unique information for the model. In addition, PCA was explored but its performance was not sufficient to justify its inclusion as a dimensionality reduction method. Overall, the feature engineering steps included those outlined in the assignment (only columns relevant to the Wednesday attack scenarios, labels outside the assignment scope removed such as Heartbleed), standardizing and rationalizing feature names, creation of the binary target variable (0 = benign traffic, 1 = DoS traffic), validating the presence of numeric only features, and storing the features in a schema. The final set of features used for training is provided in the `/data/final_features.json` file.

9.2 Justifications

Feature engineering choices were driven by the following considerations:

1. **Alignment with Best Practices** The features in the CIC-IDS2017 dataset were designed to be rich enough for ML analysis without requiring researchers to craft new features (Sharafaldin et al., 2018). Just from that literature alone, we decided to select and refine existing features rather than to generate new ones.
2. **Interpretability and Reproducibility** Consistent naming, numeric types, and adherence to the schema enables the team to reproduce results across environments and submissions. This is supported by literature emphasizing the importance of tracking transformations (Zaharia et al., 2018).
3. **Improving model stability and reducing noise** Cleaning and normalizing features reduces instability and prevents misleading patterns and errors from propagating through the rest of the pipeline. This is important in intrusion detection where there is often noisy or badly formed features - these can inflate false positive rates (Sommer & Paxson, 2010).

9.3 Status

Status: Accepted

Team Members: Fnu Syed Moosa Aleem “Moosa”, Umar Siddiqui, Nafisa Sabir, Emil Cacayan

Date: December 4, 2025

10. Processed Data Loading

10.1 Context and Technology Choice

Processed data loading refers to the process where cleaned, feature-engineered data is read into memory in preparation for modeling, evaluation, and/or deployment. In this project, the processed dataset is stored in `data/cleaned/wednesday_cleaned.csv` and loaded using `pandas` (see 7 Exploratory Data Analysis). This tool was selected mainly because it integrates naturally with downstream libraries such as `scikit-learn` and `numpy`. It also contains several operations for dealing with missing values, type casting, indexing, and other vectorized operations. In addition to loading the cleaned `.csv` file, the pipeline loads the feature schema (`final_features.json` see section 6 Reading Data) into a dictionary for later use. The schema is accessed using Python’s built-in `json` module and the team implemented loader functions in `deployment/schema_loader.py`. This design serves as a contract to prevent inconsistency between training and serving (Zaharia et al., 2019). The processed data loading in this context involves two components: loading the cleaned dataset into a `pandas` dataframe, then loading the feature schema. Both components are already integrated into the preprocessing and inference pipelines. Future adjustments may include adding further validation warnings or a more robust error handling module, but deliverables satisfy requirements as-is.

Below you can find the first 20 rows of the cleaned dataset, due to space constraints this was put here in two parts:

Destination_Port	Flow_Duration	Total_Fwd_Packets	Total_Backward_Packets	Total_Length_of_Fwd_Packets
0	443	87261	1	1
1	443	523	2	0
2	53	94304	1	1
3	53	207	2	2
4	123	69022032	2	2
5	49260	3	2	0
6	45198	99451595	6	6
7	53	61533	1	1
8	53	66577	2	2
9	443	83830	7	8
10	53	31141	1	1
11	53	99777	2	2
12	443	29810309	6	7
13	53625	238	2	0
14	53	121869	2	2
15	53	23382	1	1
16	53	27059	2	2
17	53	258	2	2
18	443	62305042	22	19
19	80	5848097	3	1

Idle_Std	...	Idle_Max	Idle_Min	Label	Attack	
0	...	0.00000	0	0	BENIGN	0
0	...	0.00000	0	0	BENIGN	0
56	...	0.00000	0	0	BENIGN	0
84	...	0.00000	0	0	BENIGN	0
96	...	0.00000	69000000	69000000	BENIGN	0
37	...	0.00000	0	0	BENIGN	0
11595	...	0.00000	99300000	99300000	BENIGN	0
42	...	0.00000	0	0	BENIGN	0
82	...	0.00000	0	0	BENIGN	0
6150	...	0.00000	0	0	BENIGN	0
38	...	0.00000	0	0	BENIGN	0
66	...	0.00000	0	0	BENIGN	0
3149	...	314803.15880	10000000	9470465	BENIGN	0
12	...	0.00000	0	0	BENIGN	0
74	...	0.00000	0	0	BENIGN	0
52	...	0.00000	0	0	BENIGN	0
68	...	0.00000	0	0	BENIGN	0
68	...	0.00000	0	0	BENIGN	0
4535	...	11269.43746	10000000	9972727	BENIGN	0
12	...	0.00000	0	0	BENIGN	0

Please visit the GitHub repo and find `data/cleaned/wednesday_clean.csv` for the full dataset. This dataset is highly dimensional so it would be infeasible to fit the entire dataframe into this ADR.

10.2 Justifications

1. **Ensures Consistency Between Training and Deployment** Loading the dataset and its schema guarantees that models use features in a fixed and validated order. The literature mentions this as one of the primary ways to combat “feature drift”, a cause of model degradation (Zaharia et al., 2018).
2. **Reproducibility and Transparency** In storing the cleaned dataset in a static file and loading it during modeling, the project aims to provide repeatable analyses. Reproducibility is a core requirement in data science and intrusion detection - small differences in feature manipulation can alter results dramatically (Sommer & Paxson, 2010).
3. **Aligns with IDS Workflows** Most research that uses this dataset loads cleaned or preprocessed variants of the dataset using CSV readers (Sharafaldin et al., 2018). `pandas` is the defacto standard that serves this purpose.
4. **Simplicity While Ensuring Deliverables** CSV storage and loading avoid overhead of database engines or complex storage systems, which are unnecessary given the size of the dataset. Simply storing the dataset in a static file reduces points of failure while remaining compatible with downstream components of this project.

10.3 Status

Status: Accepted

Team Members: Fnu Syed Moosa Aleem “Moosa”, Umar Siddiqui, Nafisa Sabir, Emil Cacayan

Date: December 4, 2025

11. Model Selection and Training

11.1 Context and Technology Choice

The modeling plan defines how the project transitions from exploratory analysis into supervised learning for binary classification of benign vs. DoS network flows. Because the CIC-IDS 2017 dataset contains high-dimensional numerical features with known redundancy and variation in scale, the team selected a combination of simple baseline models and more complex non-linear models to compare performance and interpretability.

The chosen models include:

- **Simple Models**
 - Logistic Regression
 - k-Nearest Neighbors (kNN)
 - Naïve Bayes

These models provide interpretable comparisons and establish a baseline for expected performance. Logistic regression is frequently used in intrusion detection due to historical robustness and interpretability (Ring et al., 2019), while Naïve Bayes and kNN are non-parametric baselines for this type of dataset.

- **Complex Models**
 - Decision Tree
 - Random Forest
 - Support Vector Machine (SVM)

These models were selected because DoS traffic can exhibit complex non-linear boundaries and hierarchical interactions between features (Kumar et al., 2020). Tree-based models can naturally capture threshold-based structure common in flow features, while SVM's are known to perform well on these types of datasets (Sharafaldin et al., 2018).

All models are implemented using `scikit-learn`, consistent with standard practices in modern machine learning pipelines. Data is split into training and testing sets, and preprocessing (such as standardization) is applied to the training set only.

Model selection and preprocessing design have been finalized, implementation can be found in `notebooks/04_model_training.ipynb` and `models/train_simple.py` for simple models, `models/train_complex.py` for complex models. scaling decisions and schema updates have been completed by members of the team responsible for ETL, enabling reproducible model training. Hyperparameter tuning will be limited to essential small adjustments, but may be expanded time permitting.

In addition to selecting models, the team established a list of initial hyperparameters for each classifier. These initial configurations are intentionally lighter, reflecting the time constraint of the project and recommended practices for early-stage intrusion detection and machine learning modeling. Prior research indicates that on network traffic, default or near-default hyperparameters are often sufficient for comparisons, with complexity increasing only as evidence of underfitting or overfitting is observed (Ring et al., 2019).

The project begins with `scikit-learn`'s standard parameters, with adjustments applied only where required for model convergence or numerical stability. This strategy allows the team to refine models in later evaluation stages.

11.2 Justifications

This paradigm for model selection was adopted for the following reasons:

1. Multiple Model Comparisons

Using a diverse set of model types reflects research rigor but also addresses practical uncertainty regarding which algorithm best captures the structure of DoS behavior (within this dataset). Prior studies demonstrate how attack classification performance varies significantly across algorithms due to differences in flow timing, packet dynamics, and the unbalanced behavior expected of DoS events (Sharafaldin et al., 2018; Ring et al., 2019).

In comparing the baseline and complex models, this team achieves the following: - meaningful benchmarks between models that assume linear separability vs. models that capture probabilistic behavior - investigation of non-linear relationships through trees and SVM's - evaluation of robustness to high-dimensional input - opportunities for improved generalization using bagging/ensemble methods (e.g. Random Forest)

2. Scaling Requirements

Many flow-based features (as demonstrated as well in our preliminary analyses) demonstrate high variance in magnitude (e.g. packet rat and byte statistics). Algorithms such as kNN and SVM are sensitive to this scale and require standardization to make comparisons fairly. This decision is consistent with recommended best practices for intrusion detection datasets and machine learning at large (Kumar et al., 2020).

3. Alignment with EDA

This modeling strategy was influenced by earlier exploratory findings. High feature correlation suggests that linear models may underfit but may be robust in their interpretability. Clusters observed in the multivariate analysis indicate possible non-linear structure, supporting the decision to utilize SVM and ensemble models. Outliers characteristic of DoS flows reinforce the need for models robust to extreme models (trees). Modeling decisions are therefore informed by empirical evidence rather than arbitrary selection or heuristics based on preference.

4. Reproducibility and Deployment

All chosen algorithms integrate cleanly with `scikit-learn`, simplifying deployment via FastAPI application planned for the project's final stage. This ensures consistent preprocessing, schema validation, and inference workflows.

The following hyperparameter settings were selected as initial configurations for the modeling stage:

- Logistic Regression
 - `solver='liblinear'`,
 - `max_iter=1000`
 - This solver is recommended for binary classification with smaller datasets and performs well in intrusion detection involving collinear features (Kumar et al., 2020). Increasing iteration limit mitigates the risk of non-convergence.
- k-Nearest Neighbors (kNN)
 - `n_neighbors=5`
 - `weights='uniform'`

- A small number of neighbors provides balance between sensitivity to local structure and generalization. Previous work demonstrates that kNN performs competitively on CIC-IDS flow features without extensive hyperparameter tuning (Sharafaldin et al., 2018).
- Naïve Bayes (GaussianNB)
 - Default parameters GaussianNB is widely used in intrusion detection because of its minimal assumptions and computational efficiency. Its performance is influenced primarily by scaling rather than hyperparameter tuning.
- Decision Tree
 - `max_depth=None`
 - `criterion='gini'`
 - Allowing unconstrained depth allows the model to capture smaller decision boundaries for the attacks. Later pruning may be considered if overfitting is observed.
- Random Forest
 - `n_estimators=100`
 - `max_depth=None`
 - Random Forests benefit as ensembles become larger, with a minimum of 100 trees generally recommended for stable performance despite noisy features (Ring et al., 2019).
- Support Vector Machine (SVM)
 - `kernel='rbf'`
 - `C=1.0`
 - `gamma='scale'`
 - The RBF kernel performs well on high-dimensional, non-linear datasets. The `gamma='scale'` parameter adjusts sensitivity based on variance, aligning with the skewed distribution found in EDA.

These configurations were chosen specifically to reflect properties observed in the cleaned dataset:

1. High feature variance and skew

This provides justification for standardized scaling and kernel-based SVM's.

2. Correlation structure

Tree-based methods can leverage redundant features.

3. Presence of outliers

Ensemble methods and SVM's with RBF kernels tend to be robust against outliers.

4. Lack of categorical features

Most of the features in this dataset are numeric, so hyperparameter tuning does not have to be as complex. The team rooted the selection of the hyperparameters in empirical EDA findings and established literature - and so this stage of the project is grounded in theory and observed data behavior.

11.3 Status

Status: Accepted (Model Selection)

Date: December 4, 2025

Team Members: Fnu Syed Moosa Aleem “Moosa”, Umar Siddiqui, Nafisa Sabir, Emil Cacayan

Status: Accepted (Hyperparameter Selection)

Date: December 5, 2025

Team Members: Fnu Syed Moosa Aleem “Moosa”, Umar Siddiqui, Nafisa Sabir, Emil Cacayan

12. Model Evaluation

12.1 Context and Technology Choice

The evaluation stage focused on assessing the performance of the six selected supervised classification models trained during the modeling phase: Logistic Regression, K-Nearest Neighbors (kNN), Gaussian Naïve Bayes, Decision Tree, Random Forest, and Support Vector Machine (SVM). To ensure consistency and reproducibility, all evaluations were performed using `scikit-learn`'s metrics and plotting utilities, including `classification_report`, `confusion_matrix`, generating ROC curves, and AUC calculations. These tools are widely used in machine learning research and provide standardized and interpretable metrics suitable for comparing different models under identical preprocessing conditions (Pedregosa et al., 2011).

Evaluation relied on the test set derived from the 80/20 train-test split. The test set was not used during training, allowing for unbiased evaluation of model generalization. In terms of visualization, the team generated confusion matrices and ROC curves for each model to understand misclassification and relative discriminative performance across the two labels.

Because our project evaluates binary classification within a cybersecurity context, recall and AUC were important. High recall is essential - failing to identify an attack (false negative) is significantly more harmful than incorrectly marking benign traffic as malicious (false positive). Prior work strongly emphasizes the rationale behind addressing this very asymmetrical risk, making obtaining these metrics appropriate for final model decision making (Kumar et al., 2020; Ring et al., 2019).

All evaluation steps were implemented in the notebook `notebooks/05_model_evaluation.ipynb` and output stored in: - `models/evaluation/confusion_matrices/` - `models/evaluation/roc_curves/` - `models/evaluation/metrics_tables/`

Following evaluation, across all computed metrics, the Random Forest classifier demonstrated the most robust generalization performance and achieved the strongest balance of precision, recall, and F1-score. Visual inspection of these curves demonstrated that this model dominates other models. For this project, Random Forest has been selected as the primary model for deployment in the subsequent development of inference in the deployment API. The `.pkl` file corresponding to the model has been saved to `models/saved_model/`, and the architecture outlined in section 13 assumes its use as the deployed classifier.

12.2 Justifications

The team implemented the evaluation pipeline as described above for the following reasons:

1. Consistency Across Models

In applying identical preprocessing steps (with standardization/transformation where appropriate), identical test sets, and identical evaluation metrics, we ensure that comparisons made between the models are truly meaningful. This follows best practices in machine learning development, where controlling confounding variables is central to investigations (Hastie, Tibshirani, & Friedman, 2009).

2. Appropriate Metrics for Application

As mentioned in the project summary, intrusion detection is a high-stakes classification task where false negatives (missed attacks) carry significant repercussions - much more so than false positives. Therefore, we measure recall to directly measure missed attacks, F1-score to balance precision and recall especially for this imbalanced dataset, and AUC-ROC to inform us on the separability between the two traffic types.

These choices align to the strategies used in literature, including the evaluation of this dataset which emphasize recall and ROC metrics to compare models (Sharafaldin et al., 2018).

3. Visualization for Model Interpretability

Confusion matrices allow us to diagnose systematic and logical errors, such as a model performing well on benign traffic but struggling on the higher volume DoS samples or a model misclassifying certain labels more often due to traffic similarities.

Similarly, ROC curves help to identify which models maintain strong classification across different decision thresholds. This is particularly useful for tuning in deployed systems, where thresholds may be adjusted in real time to reduce false positive rates when the system undergoes high transfer loads (Kumar et al., 2020).

4. Random Forest and SVM - Expected to Provide Strong Performance

Prior studies indicate that tree-based ensembles and kernel-based SVM's outperform linear models, especially when interactions or boundaries are non-linear (Ring et al., 2019). Our evaluation structure allows us to validate whether findings in prior literature can generalize to our machine learning pipeline.

5. Requirements for Deployment

The evaluation pipeline also allows downstream deployment possesses the model with the strongest detection capability, stable behavior across varied input data and thresholds, and consistent handling of the standardized input schema. These considerations also take into account documented constraints for deployments, where interpretability, consistency, and reduction of false-negatives are essential (Kumar et al., 2020).

12.3 Status

Status: Accepted

Date: December 6-7, 2025

Team Members: Fnu Syed Moosa Aleem "Moosa", Umar Siddiqui, Nafisa Sabir, Emil Cacayan

13. Model Deployment

13.1 Context and Technology Choice

The deployment stage of this project focuses on making the trained DoS classifier operational so that predictions can be generated through an API. We chose to adopt a simple deployment approach, where the model is saved to a .pk1 file and can be loaded and used for predictions in a subsequent deployment API. The architecture we chose was built around a FastAPI application. In this architecture, incoming network-flow samples are validated, transformed, and then passed to the model for inference, returning a binary prediction for benign or attack traffic.

The deployment pipeline is organized within this project's `deployment/` directory and includes the following components: 1. `app.py`: Implements `/health`, `/version`, and `/predict` endpoints for the API.

It loads the trained model and uses the preprocessing pipeline to transform inputs before inference. 2. `schema_loader.py`: Loads the `final_features.json` schema that defines expected input features, types, and transformations. The schema generated earlier ensures consistent preprocessing and feature ordering. 3. `preprocessing.py`: Validates input samples, checks for missing or unexpected fields, enforces data types, converts features to numeric values, and applies scaling transformations using statistics stored in the schema. 4. `final_features.json`: Encodes the feature set, expected preprocessing parameters, and statistics used during model training. This helps to guarantee reproducibility across the entire inference pipeline.

This modular approach aligns with design patterns in machine learning deployment, where preprocessing and inference logic is ideally consistent between training and production environments (Raschka et al., 2022; Zaharia et al., 2018). The architecture is intentionally simple to make local testing easier, but could be further expanded and perhaps containerized.

This architecture has been implemented and tested locally, with all endpoints operational. Preprocessing validation, schema loading, and app architecture are implemented in `deployment/preprocessing.py`, `deployment/schema_loader.py`, and `deployment/app.py`, respectively. Testing both locally and with Postman are performed/demonstrated in `notebooks/06_deployment.ipynb`.

13.2 Justifications

We had several considerations that led us to choose this deployment architecture:

1. **FastAPI for the API Layer** We chose this API because of its strong performance, intuitive syntax, and built-in validation, which is commonly used in production ML systems. One of the main advantages of FastAPI is its low latency and ability to perform asynchronously - which adds to the speed at which inference occurs which is key in many network traffic applications (Ramírez & Lanusse, 2022).
2. **Schema for Consistency and Reproducibility** Using the `final_features.json` schema offers quite a few advantages. The first is consistency - we can guarantee that the same feature transformations are applied in training and deployment. Additionally, the schema serves as documentation including statistics, types, and metadata which makes our pipeline more transparent - making it easy to audit or debug.
3. **Modularity** Use of the schema allows for changes to the features without restructuring the architecture, which is essential for the maintainability and portability of our code. This is not only a machine learning paradigm, but a general software development best practice.

13.3 Status

Status: Accepted

Date: December 6-7, 2025

Team Members: Fnu Syed Moosa Aleem “Moosa”, Umar Siddiqui, Nafisa Sabir, Emil Cacayan

14. Concluding Remarks and Project Summary

14.1 Context and Technology Choice

The purpose of this project was to design and implement a machine learning pipeline to distinguish Denial of Service (DoS) attacks from benign network traffic using the CIC-IDS2017 Wednesday dataset. The project required integration of multiple system components, including data extraction, cleaning, schema construction, exploratory data analysis, model training, evaluation, and deployment - into a workflow closely adhering to modern machine learning best practices.

For implementation, the team made a few deliberate technology choices. Python served as the primary due to its large collection of data processing libraries (and its extensive use in COMP479). `pandas` and `numpy` facilitated manipulation and transformation of loaded datasets, simplifying the handling of the highly dimensional network traffic data. `scikit-learn` was selected for modeling because of its suitability for structured data. FastAPI was used for deployment given its speed, simplicity, and built-in validation. A custom schema architecture ensured consistency between training and deployment, preventing feature-ordering and type mismatches often seen in production ML systems (Zaharia et al., 2018). Together, these choices allowed us to create a robust, well-documented, and collaborative approach for first identifying harmful traffic problems and at large implementing, testing, and deploying machine learning models.

14.2 Justifications

1. **Ensuring Pipeline Reliability and Reproducibility** Each stage was designed to guarantee reproducibility. The schema-driven approach ensures that any new dataset or inference request is validated against the same feature constraints used during training, which is as mentioned previously a best practice in machine learning (Zaharia et al., 2018).
2. **Supporting Interpretability and Diagnostics** The modular structure of this pipeline enabled the team to assess data quality, detect outliers and anomalies, and predict downstream effects of upstream decisions.
3. **Addressing the Nature of DoS Attack Traffic** DoS attacks produce distinctive flow patterns (abnormally high packet rates and skewed distributions) which justified the project’s modeling choices. Random Forests and logistic regression are frequently used for this type of data because of their relatively low tendency to overfit noisy features (Kumar et al., 2020).
4. **Facilitating Deployment** The architecture for preprocessing and inference were packaged within FastAPI, meaning the final inference system is usable outside the environment of the jupyter notebooks used for the analysis portions of this project. This is a proof-of-concept for real time security monitoring systems (Sharafaldin et al., 2018).
5. **Clean Separation of Concerns for Productivity and Maintainability** The modular structure of the project lent itself to separation of different tasks/concerns, which mirrors software engineering best practices. This also streamlined development of all the stages of our project, as team members could work independently of one another.

14.3 Status

Status: Accepted **Date:** December 10, 2025 **Team Members:** Fnu Syed Moosa Aleem “Moosa”, Umar Siddiqui, Nafisa Sabir, Emil Cacayan

15. References

- Breck, E., Polyzotis, N., Roy, S., Whang, S. E., & Zinkevich, M. (2017). The ML test score: A rubric for production-ready machine learning systems. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. <https://doi.org/10.1145/3097983.3098181>
- Breck, E., Polyzotis, N., Roy, S., Whang, S. E., & Zinkevich, M. (2019). Data validation for machine learning. In *Proceedings of Machine Learning and Systems 1* (pp. 1–16). *MLSys*. proceedings.mlsys.org
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>

- Huyen, C. (2022). Designing machine learning systems: An iterative process for production-ready applications. O'Reilly Media.
- Khaitan, S. K., & McCalley, J. D. (2015). Design techniques and applications of cyberphysical systems: A survey. *IEEE Systems Journal*, 9(2), 350–365. <https://doi.org/10.1109/JSYST.2014.2322503>
- Kumar, P., Singh, A., & Sharma, R. (2020). Denial-of-Service attacks and their mitigation techniques: A survey. *International Journal of Network Security*, 22(4), 634–648.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- Rauf, I., Petre, M., Tun, T., Lopez, T., Lunn, P., Van der Linden, D., Towse, J., Sharp, H., Levine, M., Rashid, A., & Nuseibeh, B. (2021). The case for adaptive security interventions. *ACM Transactions on Software Engineering and Methodology*, 31(1), Article 9. <https://doi.org/10.1145/3471930>
- Ring, M., Wunderlich, S., Scheuring, D., Grödl, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. *Computers & Security*, 86, 147–167.
- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSP 2018—Proceedings of the 4th International Conference on Information Systems Security and Privacy*, 108–116. <https://doi.org/10.5220/0006639801080116>
- Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. *2010 IEEE Symposium on Security and Privacy*, 305–316. <https://doi.org/10.1109/SP.2010.25>
- Tiangolo, S. (2018). FastAPI documentation. <https://fastapi.tiangolo.com>
- van der Aalst, W. (2022). Process mining and data quality: A review of challenges and opportunities. *ACM Computing Surveys*, 55(7), 1–39. <https://doi.org/10.1145/3527154>
- Zaharia, M., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., & Stoica, I. (2018). An overview of machine learning pipelines in production. *Communications of the ACM*, 61(4), 36–45. <https://doi.org/10.1145/3184402>